Product Spec

Opportunity

Timing of requests making their way into HealthSource from **fax, email, print/scan request letters, and uploaded request letters** is taking **up to several hours** from the time of import until the request is available to work in HealthSource.

This delay is driving **operations teams to create manual workarounds**, process requests outside of HealthSource, and in some cases **impacting patient care and turnaround time**.

While the team made meaningful progress in Q3 improving **fax and email intake**, this next phase focuses on **expanding those improvements to manual upload and print/scan pathways** — continuing our platform-level acceleration of intake performance.

This initiative is foundational as we look ahead to **2026**, when HealthSource will undergo workflow modernization for **uploading request letters and work management**. Establishing consistent, observable, and fast intake behavior now ensures those future experiences are built on a reliable, high-performance foundation.

Goal

Reduce the time from any intake source — fax, email, manual upload, or print/scan — to e-request ID creation from **hours to minutes**, while introducing **real-time observability and alerting** for intake performance.

This work is an extension of the Q3 intake acceleration effort, expanding scope and establishing uniform speed and reliability across all ingestion points.

Impact

- Restores **HealthSource as the single system of record** by reducing reliance on external workarounds.
- Improves **turnaround time (TAT)** and adherence to operational SLAs.
- Enables **faster triage and patient request visibility** for time-sensitive workflows.
- Establishes a **Datadog-based intake monitoring and alerting dashboard**, similar to STORK, with metrics (p99, p95) to proactively detect latency and SLA risk.
- Provides the necessary telemetry and performance discipline for upcoming 2026 workflow rewrites.

Hypotheses

1. **By instrumenting intake and upload workflows**, we can measure and identify service-level agreement risks and more effectively prioritize potential remediations.
2. **By addressing top sources of intake lag**, we can improve turnaround-time SLA adherence and create efficiencies for field associates who are trying to upload and work urgent requests.
3. **By applying consistent orchestration and telemetry across all intake types**, we can reduce variability in processing times and establish a reliable intake performance baseline across the platform.

Each hypothesis focuses on measurable impact, not prescribed implementation — the goal is to surface the right data, target the most impactful latency sources, and validate system-wide performance gains.

Press Release (Future State Vision)

FOR INTERNAL RELEASE – HealthSource Platform Team

HealthSource now processes every incoming request — from fax, email, print, or upload — in near-real time.

What once took hours now happens in minutes. Requests are immediately available to work, without manual intervention or workarounds.

Operations teams can rely on the system again, confident that urgent requests flow through quickly and predictably.

A new intake performance dashboard provides complete transparency and alerting, ensuring that any slowdown is visible and correctable in real time.

This milestone not only accelerates today's intake but also lays the foundation for our 2026 modernization of request letter upload and work management workflows.

Pre-Mortem: How This Could Go Sideways

- **Instrumentation gaps:** We fail to measure the full pipeline, making true bottlenecks invisible.
- **Fragmented implementation:** Different intake paths improve at different rates, causing inconsistency.

- **Performance at the expense of accuracy:** Speed improvements result in duplicate or incomplete e-request IDs.
- **Vendor or infrastructure dependency:** Latency persists due to limits in external NLP/OCR services.

Mitigation: implement observability first, align all intake sources under a shared monitoring model, and validate improvements using end-to-end timing metrics.

Prioritized Scope

In Scope

- Extend intake performance acceleration work to manual and print upload pathways.
- Instrument all intake routes with detailed telemetry (DataDog or equivalent).
- Establish real-time alerting for intake timing anomalies.
- Evaluate and address top latency contributors in orchestration and NLP flows.

Out of Scope

- UI/UX or workflow redesigns (planned separately for 2026).
- Modifications to downstream processes after e-request ID creation.

Acceptance Criteria

As a platform operations user, I want requests received through any intake method — fax, email, manual upload, or print — to generate an e-request ID quickly and consistently, so that I can begin processing immediately without external workarounds and maintain SLA performance.

Feature Type

Feature — Platform performance initiative involving intake, orchestration, and telemetry services.

User-Facing?

No — backend-focused; no end-user UI changes.

Dependencies & Constraints

- Platform Engineering
- NLP Infrastructure
- Data Engineering (for observability, DataDog integration)
- Prior fax/email intake improvements as baseline work

Risks & Mitigation

- **Fragmented parity:** Roll out uniformly to maintain consistent behavior across intake types.
- **Monitoring noise:** Calibrate thresholds carefully to prevent false alerts.
- **Quality regression:** Validate accuracy and ID completeness after any acceleration change.

Data & Analytics Plan

- Track event timestamps: import → intake processing start → e-request ID created.
- Capture average, p95, and p99 times per intake type.
- Create a DataDog dashboard with SLA alerts and historical trend visibility.
- Measure improvement over current baselines and surface anomalies automatically.

Open Questions

- What are the specific latency thresholds we want to target for alerting (e.g., >15 min p95)?
- Should the same alert logic apply to all intake types or vary by source?
- How do we align intake metrics with broader SLA reporting in 2026 roadmap systems?

Release Strategy & Iterations

Phase 1 – Discovery:
Baseline current intake performance across manual and print paths; confirm alignment with existing fax/email monitoring.

Phase 2 – Instrumentation:
Implement telemetry and dashboarding; validate data accuracy.

Phase 3 – Optimization:
Target top latency contributors and validate SLA improvements.

Phase 4 – Rollout:
Enable full monitoring, alerts, and reporting across all intake pathways; validate performance stability.

Tech Spec

Purpose

The technical spec / architectural design document helps align our teams on how we will achieve the Product Spec. There are many ways to solve a problem, and this document describes how we'll balance many competing priorities to achieve a solution that delivers optimal business value.

Section Owner:

The engineer(s) assigned to the discovery and scoping of this feature are responsible for owning the content in this section. We invite others to provide input, feedback, and questions on this section.

Order of Operations

The product and engineering teams collaborate on this document to provide increasing levels of clarity around the requirements and best approach that provides the most business value for each situation.

This process will take several distinct tickets and meetings. The goal is to provide initial visibility to the business about risks, opportunities, and costs. As we hone in on identifying exactly what we build and how much time we have available, the later stages involve finalizing the actual technical design that will be built.

While the whole document is needed for a complete review, a "first pass" sizing should always include:

- Design Goals
- One or more proposed architectural designs
- Size Estimate
- Confidence
- Risks/Mitigations table
- Rollout mechanics

Subsequent passes should increase the detail level (accuracy and precision) of the above sections.
During the final pass discovery stage and prior to the kickoff meeting, all remaining sections should be completed as best as possible.

Note: the kickoff meeting must always include Tyler Beach to help satisfy requirements for CASTLE (https://datavant.atlassian.net/wiki/spaces/PS/pages/1508179991/CASTLE+for+Engineers)

System Context

Why: A simple explanation that lists the system components/domains involved and their current state so that readers can be oriented for the remaining context in the document.
What: 1 or 2 paragraphs describing the state of the system as it pertains to the requested change. Links to any previous architecture diagrams or other artifacts helpful in understanding this area.

Design Goals

Why: In order to help align our teams on business value and tradeoffs, we want to explicitly state our design goals that were prioritized when developing the proposed designs.
What: A paragraph or some bullet points of how we balanced such competing forces as, for example:

- Do we build it ourselves, partner with someone, or buy a solution?
- How do we balance quality, cost, and time so that we achieve an optimal business outcome?
- Is a human powered/manual approach better or does automation yield a better balance for the business (short term vs. long term)?
- Is phasing necessary to minimize risk, and what is the appropriate complexity to try to wrangle in the proposed design?
- How important is developer experience/velocity vs. reusability and maintainability of a given change? What is best for the business right now and in the future?
- We are lacking good test coverage in many places in HealthSource. How risky is the change, and how much risk mitigation is necessary as an implicit part of designing the solution?

Proposed Architectural Designs

Why: To help us align our teams on what the change would actually look like, how simple/complex it is, and what pieces of the system are in play for a given change. This is what we're sizing.
What: A collection of text, system diagrams, states, or flows to show what the system would do if we approached the problem a certain way. The description should be in enough detail that the size and complexity of the change can be ascertained. In earlier phases of our design lifecycle, note the options or tradeoffs in each area so that we can collectively align on competing design goals. In later stages of the design, we do not desire ambiguity in this section and want only a single design solution to recommend.

Size Estimate

Why: In order to figure out the right mix of changes for a quarter and also to figure out who are the right people to make the changes, we need to understand an estimate of total cost.
What: Preferably, an estimated number of engineers for an estimated number of total sprints. If we need specific people or teams involved, note that in the cost since dependencies are important contributors to overall cost. Alternatively, hours or t-shirt sizes are acceptable if this is the most accurate we can be. High accuracy is expensive, but we don't want to be irresponsible by being too imprecise.

Confidence

Why: Every project has a unique risk profile and unique tradeoffs/challenges in order to deliver ideal business outcomes, and so we want to try to capture a confidence measure.
What: One or two sentences describing how confident we are about our size estimate.

Risks/Mitigations

Why: Even with the best design, some risks are always present. Call out any open questions or concerns as well as any risks we are well aware of and how we'll defend against them.
What: A table with a list of risks and what we're going to do to reduce the risk (or how the existing design proposed defends against the risk).

Risk	**Mitigation**

Rollout Mechanics

Why: Delivering business value can rarely/wisely be accomplished in a single step. We want to understand the complexity and steps needed to deliver value to customers, what is delivered in each phase, and how what might need to be communicated at each step of the process.
What: A series of steps, an outline, or a sequence diagram showing how we'd release the changes, what things might need to be migrated/released in each stage, and how we'd rollout the functionality to users. The focus is on what technical changes are needed and what business value users will be delivered during each step.

Note any important system or functional criteria that should be true before advancing to the next phase.
If rollback is for some reason not possible in a step, note that here so we can understand the implications.
This section is intended to be complementary to the **Release Strategy** tab.

Security Considerations and Implications

Why: We want to make sure that all changes take security implications into account and that the risk of those changes is called out.
What: A paragraph or bullet point list describing the known security risks or places where the change interacts with important data, encryption, or auth. We follow the CASTLE process, and this section is not meant to replace that. Tyler Beach is responsible for assessing overall security risk.

Monitoring, Alerting, and Measuring

Why: All software changes need to be supported by the team, and this means having a good monitoring and alerting strategy for the change. Likewise, we want to make sure we're measuring both platform level metrics as well as business level metrics.
What: A bullet point list of what things we'd measure, where we'd measure it (Datadog, Sigma, or a manual query) and alert on. The severity/response plan of the metric should be noted so that we can understand what things wake people up vs. just are used to track adoption.

Testing

Why: All changes must be tested, but not all changes have the same risk level when developing tests. This section helps us align on how deep we should go on which areas of testing the change.
What: A list of specific tests or strategies that we will use to verify the changes, and if applicable, which parts might be done manually vs. in an automated fashion.

Dependencies

Why: Some changes will require new permissions, access, or important work from other teams in order to be complete. We should note that here.
What: A bullet point list of external dependencies that we will need in order for this project to be successful.

Release Strategy

Release Strategy & Iterations

Maturity Level (for Feature or Major Feature):

- **Experiment** – Small-scale (<5 customers) for early hypothesis testing
- **Alpha** – Initial test with up to 10 customers
- **Beta** – Scaled test with up to 100 customers
- **General Availability (GA)** – Broad release to all customers

Normally, features progress through alpha → beta → GA. Skipping steps requires justification.

Rollout Toggles

- Indicate whether a toggle is needed
- PMs should use toggles unless clearly unnecessary
- Toggles, when needed, should ideally be labeled with ticket numbers so that complexity can be manageable long term for our future selves.

Iterations

- Define phases (alpha, beta, GA)
- Include rough timelines
- Note scope and exclusions for each stage